pytest Daily Cheat Sheet

Run · Select · Debug · Assert · Fixtures · Parametrize · Markers · Config

1 · Run tests

```
pytest                          # all tests
pytest -v                       # verbose: each test + result
pytest tests/test_login.py      # one file
pytest test_login.py::test_valid  # one test
pytest test_api.py::TestUsers   # one class
pytest tests/api/               # one folder
```

2 · Select tests

```
pytest -k "login and not slow"  # by name keyword
pytest -m smoke                 # by marker
pytest -m "smoke or regression"
pytest --collect-only -q        # list tests, don't run
```

3 · Debug & re-run

```
pytest -x             # stop at first failure
pytest --lf           # re-run Last Failed only
pytest --ff           # Failed First, then the rest
pytest -s             # show print() output
pytest -vv            # full diff of lists/dicts
pytest --tb=short     # shorter tracebacks
pytest --pdb          # debugger at the failure
pytest -rA            # summary of all results
pytest --setup-show   # fixture setup/teardown order
```

4 · Speed & reports

```
pytest -n auto                  # parallel   (pytest-xdist)
pytest --durations=5            # 5 slowest tests
pytest --junitxml=results.xml   # CI report (built in)
pytest --html=report.html       # HTML       (pytest-html)
pytest --reruns 2               # retry flaky (rerunfailures)
pytest --cov=src                # coverage   (pytest-cov)
```

5 · Assertions (compare & verify)

```
assert status == 200                # equal
assert status != 404                # not equal
assert "admin" in roles             # in list / substring
assert user.get("manager") is None  # None: use "is"
assert isinstance(uid, int)         # type
assert body == {"id": 7, "ok": True} # dict content
assert 0.1 + 0.2 == pytest.approx(0.3)   # floats
assert code in (200, 201), f"Got {code}" # custom message

with pytest.raises(ValueError, match="invalid"):
    parse_id("x")                   # passes only if it raises
```

6 · Fixtures (setup / teardown)

```
@pytest.fixture                 # scope="function" (default)
def user():
    return {"name": "Hari"}

@pytest.fixture(scope="session")   # or class / module
def browser():
    b = launch()                   # setup
    yield b                        # ← test runs here
    b.close()                      # teardown (always runs)

def test_home(browser, user):      # injected by NAME
    assert user["name"] == "Hari"
```

Built-in	Use
`tmp_path`	Temp folder per test
`monkeypatch`	Set env vars / patch (auto-restored)
`capsys`	Capture print output
`request`	Test info, `request.param`

conftest.py = shared fixtures for the folder (no import needed)

7 · Parametrize (data-driven)

```
@pytest.mark.parametrize("user, pwd, expected", [
    ("admin", "Admin@123", "dashboard"),
    ("admin", "wrong",     "error"),
], ids=["valid", "wrong-password"])          # readable names
def test_login(user, pwd, expected):
    assert login(user, pwd) == expected

pytest.param(0.1, 0.2, 0.3,                  # mark ONE row
             marks=pytest.mark.xfail(reason="float"))

@pytest.mark.parametrize("browser", ["chromium", "firefox"])
@pytest.mark.parametrize("device", ["desktop", "mobile"])
# stacked decorators → 2 × 2 = 4 runs
```

8 · Markers

```
@pytest.mark.smoke              # custom tag → -m smoke
@pytest.mark.skip(reason="not ready")
@pytest.mark.skipif(sys.platform == "win32", reason="Linux")
@pytest.mark.xfail(reason="JIRA-123")  # expected to fail
```

9 · pytest.ini (default options)

```
[pytest]
addopts = -v --tb=short -rfE
testpaths = tests
markers =
    smoke: quick critical tests
    regression: full suite
```

10 · Discovery rules & TestNG → pytest

Files `test_*.py` / `*_test.py` · functions `test_*` · classes `Test*` (no `__init__`)

TestNG / JUnit	pytest
`assertEquals(a, b)`	`assert a == b`
`@BeforeMethod` + `@AfterMethod`	fixture with `yield`
`@BeforeSuite` / `@BeforeClass`	`scope="session"` / `"class"`
`@DataProvider`	`@pytest.mark.parametrize`
groups	markers + `-m`
`testng.xml`	`pytest.ini` / `pyproject.toml`
`assertThrows`	`pytest.raises`

🧠 Memory hooks

-k keyword · **-m** marker · **-x** exit on first fail · **-s** show prints · **--lf** last failed
Floats → approx · Errors → raises · Everything else → assert
yield = setup above, test here, teardown below · **Scope F-C-M-S**: plate → table → kitchen → restaurant
parametrize = names in one string, rows as tuples · **conftest.py** = shared toolbox